

EE-170L Computer Programming Lab

Laboratory Session 10

More About Functions in C++

Objective

The students will learn and apply the

- Function Call by Value
- Function Call by Reference
- Default Values in Parameters

1 CallingFunction: Call By Value

Call by value is used when whenever the called function does not need to modify the value of the caller's original variable. In this case a copy of the variable is passed to the function and when the function changes its values then it has no effect on the value of the variable in the main function. Note that & (ampersand) is called "**addressof**" operator. It is used to mention the address of a variable. For example `int &num` describes the address of the variable `num`.

Example:

```
1  #include <iostream>
2  using namespace std;
3
4  void func(int); // function prototype
5  int main()
6  {
7      int i = 10;
8      func(i);
9      cout << "The value of i remains " << i << endl;
10     return 0;
11 }
12 // function definition
13 void func(int i)
14 {
15     i = i + 10;
16     cout << "Inside function , the value of i changed to " << i << endl; 17
17 }
```

LabCodes/lab11p01.cpp

Output:

```
Inside function, the value of i changed to 20
The value of i remains 10
```

Example:

```
1  #include <iostream>
2  using namespace std;
3
4  void swapping(int, int); // function prototype
5  int main()
6  {
7      int x = 50, y = 70;
8      cout << "Before calling function: values are: ";
9      cout << "x = " << x << " and " << "y = " << y << endl;
10     swapping(x, y);
11     cout << "After calling function: values are: ";
12     cout << "x = " << x << " and " << "y = " << y << endl;
```

```
13         return 0;
14     }
15     // function definition
16     void swapping(int x1, int y1)
17     {
18         int z1;
19         z1 = x1;
20         x1 = y1;
21         y1 = z1;
22         cout << "Values inside function: ";
23         cout << "x1 = " << x1 << " and " << "y1 = " << y1 << endl;
24     }
25 }
```

LabCodes/lab11p02.cpp

Output:

Before calling function: values are: x = 50 and y = 70
Values inside function: x1 = 70 and y1 = 50
After calling function: values are: x = 50 and y = 70

2 Calling Function: Call By Reference

If you are calling by reference it means that compiler will not create a local copy of the variable which you are referencing to. It will get access to the memory where the variable saved. When you are doing any operations with a referenced variable you can change the value of the variable.

Example:

```
1  #include <iostream>
2  using namespace std;
3
4  void swapping(int &, int &); // function prototype
5  int main()
6  {
7      int x = 50, y = 70;
8      cout << "Before calling function: values are: ";
9      cout << "x = " << x << " and " << "y = " << y << endl;
10     swapping(x, y);
11     cout << "After calling function: values are: ";
12     cout << "x = " << x << " and " << "y = " << y << endl;
13     return 0;
14 }
15 // function definition
16 void swapping(int &x1, int &y1)
17 {
18     int z1;
19     z1 = x1;
```

```
20     x1  =  y1;
21     y1  =  z1;
22 }
```

LabCodes/lab11p03.cpp

Output:

Before calling function: values are: x = 50 and y = 70
After calling function: values are: x = 70 and y = 50

Passing by reference is also an effective way to allow a function to return more than one value. For example, here is a function that returns the previous and next numbers of the first parameter passed.

3 Default Values in Parameters

When declaring a function we can specify a default value for each of the last parameters. This value will be used if the corresponding argument is left blank when calling to the function. To do that, we simply have to use the assignment operator and a value for the arguments in the function declaration. If a value for that parameter is not passed when the function is called, the default value is used, but if a value is specified this default value is ignored and the passed value is used instead.

Example:

```
1  #include <iostream>
2  using namespace std;
3
4  // function definition
5  int divide(int a = 8, int b = 2)
6  {
7      int r;
8      r = a / b;
9      return (r);
10 }
11
12 int main()
13 {
14     cout << divide();
15     cout << endl;
16     cout << divide(12);
17     cout << endl;
18     cout << divide(63, 7);
19     return 0;
20 }
```

LabCodes/lab11p04.cpp

Output:

4
6
9

Lab11: Lab Tasks

1. Function with Parameters

- Declare a function called "multiply" that takes two integer parameters: "num1" and "num2".
- Inside the function, calculate the product of the two numbers.
- Display the result within the function.
- Call the function from the main program, passing two numbers, to perform the multiplication.

2. Function with Return Value

- Declare a function called "getSquare" that takes an integer parameter: "number".
- Inside the function, calculate the square of the number.
- Return the result.
- Call the function from the main program, passing a number, and display the square.